GDC BANK

GESTION ET DÉVELOPPEMENT DU CAPITAL

BANKING MANAGEMENT APPLICATION

A Desktop Management System

PROJECT DOCUMENTATION

Project Documentation and Results

Python • Tkinter • MySQL

A desktop application for customer, account, and transaction management.

Prepared by: ACHRAF LAHLOU, ABDERRAHIM BOURICH, ILYASS MEKAOUI

Academic year: 2026–2027

Project type: Python desktop application

Technologies: Python, Tkinter, MySQL

Learn. Develop. Manage.

Contents

1 General Introduction **2**
1.1 Project Context . . . 2
1.2 Project Scope . . . 2

2 Project Objectives **2**
2.1 Main Objective . . . 2
2.2 Specific Objectives . . . 2

3 Technologies Used **3**
3.1 Why These Technologies? . . . 3

4 Database Design **3**
4.1 Creating the Database . . . 3
4.2 Customers Table . . . 3
4.3 Accounts Table . . . 3
4.4 Transactions Table . . . 4
4.5 Naming the Accounts Foreign Key . . . 4
4.6 Database Validation Constraints . . . 4
4.7 Database Relationships . . . 5

5 Application Architecture **5**
5.1 Role of Each Layer . . . 5

6 Python–MySQL Connection **5**
6.1 Database Configuration . . . 5
6.2 Reusable Query Function . . . 6

7 Main Python Classes and Operations **6**
7.1 Customer Class . . . 6
7.2 Opening an Account . . . 7
7.3 Deposit and Withdrawal . . . 7
7.4 Transfers . . . 8
7.5 Amount Validation . . . 8

8 Results: Application in Operation **8**
8.1 Dashboard . . . 8
8.2 Customer Management . . . 9
8.3 Account Management . . . 9
8.4 Transaction Management . . . 10

9 Conclusion **10**

1 General Introduction

Project Overview

GDC BANK is an educational desktop banking-management simulator. It manages customer records, bank accounts, and simulated financial transactions through a graphical interface. The application is developed using Python, Tkinter, and MySQL.

1.1 Project Context

Managing customer information and bank accounts requires accurate record keeping, clear organization, and consistent data.

A desktop application can bring these operations together in one interface, reducing repetitive tasks and making information easier to find.

GDC BANK addresses this need through a structured application that allows users to manage customer records, associate accounts with customers, and maintain a history of simulated financial transactions.

1.2 Project Scope

The application is designed for educational and demonstration purposes. It illustrates the use of Python programming, graphical interfaces, relational databases, and transaction management.

Important Scope Limitation

GDC BANK is a simulation project. It is not a real banking system and is not connected to banking networks or real payment services.

2 Project Objectives

2.1 Main Objective

Build a functional desktop management application connected to a relational MySQL database.

2.2 Specific Objectives

- Register, display, search, update, and delete customer records.
- Open and display customer accounts.
- Maintain the balance of each account.
- Simulate deposits, withdrawals, and transfers between accounts.
- Store and filter transaction history.
- Display statistics on a dashboard.
- Export displayed tables to CSV files.
- Validate inputs and handle database errors.

Expected Result

A desktop application in which customer, account, and transaction operations work together through a consistent relational database.

3 Technologies Used

Technology	Role	Objective
Python	Application logic	Validate information and coordinate operations.
Tkinter	Graphical interface	Create windows, forms, tables, buttons, and dialogs.
MySQL	Database management	Store customers, accounts, and transactions.
mysql-connector-python	Python–MySQL connector	Execute SQL statements from Python.
CSV	Export format	Save displayed table data in spreadsheet-compatible files.

3.1 Why These Technologies?

- **Python** provides the programming logic and connects the different parts of the application.
- **Tkinter** provides the graphical components used to interact with the application.
- **MySQL** stores information in related tables and maintains data integrity.
- **CSV** makes it possible to export information for use in spreadsheet software.

4 Database Design

4.1 Creating the Database

```
CREATE DATABASE GDC_Bank;
USE GDC_Bank;
```

Objective: `CREATE DATABASE` creates the database. `USE` selects it so that the following table definitions are created inside `GDC_Bank`.

4.2 Customers Table

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTO_INCREMENT,
    full_name VARCHAR(40) NOT NULL,
    email VARCHAR(60) UNIQUE,
    created_at DATE NOT NULL
);
```

This table stores the identity and contact information of each customer.

- `id`: unique customer identifier and primary key.
- `full_name`: customer's full name.
- `email`: customer's email address; values must be unique.
- `created_at`: customer registration date.

4.3 Accounts Table

```
CREATE TABLE accounts (
    id INT PRIMARY KEY AUTO_INCREMENT,
    id_customers INT NOT NULL,
    balance DECIMAL(15,2) NOT NULL DEFAULT 0.00,
    created_at DATE NOT NULL,
    FOREIGN KEY (id_customers) REFERENCES customers(id)
);

ALTER TABLE accounts
RENAME COLUMN id_customers TO customers_id;
```

This table stores the accounts and their balances. The foreign key connects each account to an existing customer.

- `id`: unique account identifier.
- `customers_id`: identifies the account owner.

- `balance`: current account balance.
- `created_at`: account opening date.

Relationship

One customer can own several accounts, but each account is associated with one customer.

4.4 Transactions Table

```
CREATE TABLE transactions (
    id INT PRIMARY KEY AUTO_INCREMENT,
    account_id INT NOT NULL,
    type VARCHAR(20) NOT NULL,
    amount DECIMAL(15,2) NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (account_id) REFERENCES accounts(id)
);
```

This table preserves the history of operations for each account.

- `id`: unique transaction identifier.
- `account_id`: account concerned by the operation.
- `type`: type of transaction.
- `amount`: amount involved in the operation.
- `created_at`: date and time of the transaction.

The transaction types used by the application are `deposit`, `withdrawal`, `transf_out`, and `transf_in`.

4.5 Naming the Accounts Foreign Key

```
ALTER TABLE accounts
DROP FOREIGN KEY accounts_ibfk_1;

ALTER TABLE accounts
ADD CONSTRAINT fk_account_customers
FOREIGN KEY (customers_id) REFERENCES customers(id);
```

The first statement removes the original foreign-key constraint. The second adds it again with the explicit name `fk_account_customers`. A named constraint is easier to identify when the schema is maintained.

4.6 Database Validation Constraints

```
ALTER TABLE customers
ADD CONSTRAINT chk_email
CHECK (email LIKE '\_%@\_%._%');

ALTER TABLE customers
ADD CONSTRAINT chk_name
CHECK (CHAR_LENGTH(TRIM(full_name)) >= 2);

ALTER TABLE accounts
ADD CONSTRAINT chk_balance
CHECK (balance >= 0);

ALTER TABLE transactions
ADD CONSTRAINT chk_amount
CHECK (amount > 0);

ALTER TABLE transactions
ADD CONSTRAINT chk_type
CHECK (type IN
('deposit','withdrawal','transf_out','transf_in'));
```

Explanation of the constraints:

- `chk_email`: checks the email against the supplied `LIKE` pattern.
- `chk_name`: `TRIM` removes surrounding spaces; `CHAR_LENGTH` checks that at least two characters remain.
- `chk_balance`: prevents a negative balance.
- `chk_amount`: requires a strictly positive amount.

- `chk_type`: restricts the type to the four values used by the application.

4.7 Database Relationships

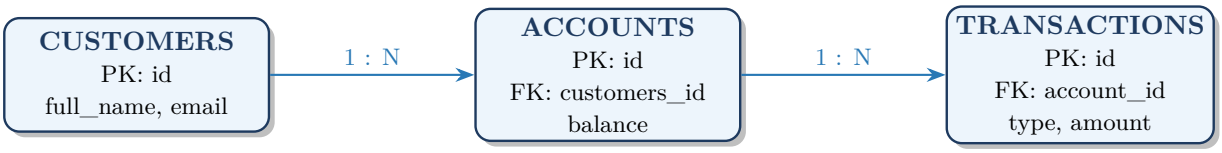


Figure 1: Conceptual representation of the database relationships

Database Structure

The database contains three main tables: **customers**, **accounts**, and **transactions**. Primary keys identify records, while foreign keys connect related records and maintain referential integrity.

5 Application Architecture

The application is organized into three logical layers.

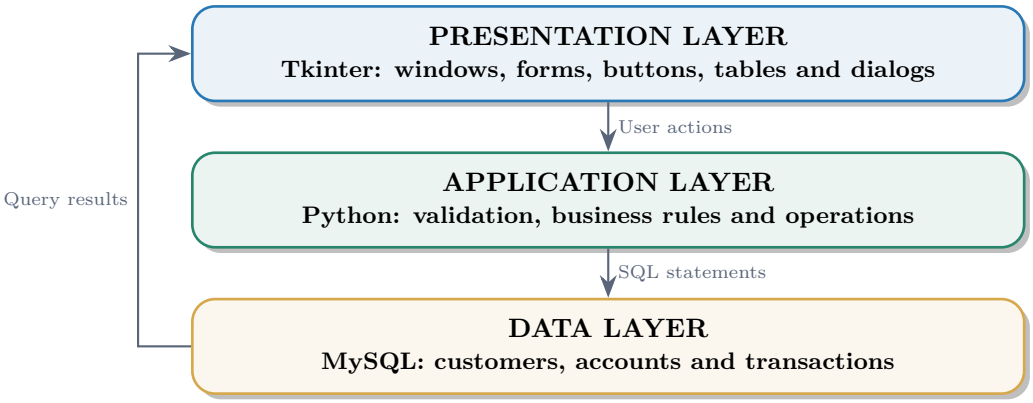


Figure 2: Three-layer architecture of GDC BANK

5.1 Role of Each Layer

- **Presentation layer:** displays the interface and receives user actions through Tkinter.
- **Application layer:** processes requests, checks information, and applies the application's rules.
- **Data layer:** stores and retrieves information using MySQL.

This organization separates the graphical interface from the application logic and database operations.

6 Python–MySQL Connection

6.1 Database Configuration

```python
import mysql.connector
from mysql.connector import Error

DB_CONFIG = dict(
    host="localhost",
    user="root",
    password="YOUR_MYSQL_PASSWORD",
    database="GDC_Bank",
    port=3306
)

def get_connection():
    return mysql.connector.connect(**DB_CONFIG)
```

Explanation:

- `mysql.connector` is the Python connector for MySQL.

- `DB_CONFIG` groups the connection settings.
- `host` identifies the server; `localhost` means the server is on the same computer.
- `user` and `password` identify the MySQL account.
- `database` selects the GDC BANK database.
- `get_connection()` opens and returns a connection.

6.2 Reusable Query Function

```
def run_query(sql, params=(), fetch=False):
    conn = get_connection()
    try:
        cur = conn.cursor()
        cur.execute(sql, params)

        if fetch:
            return cur.fetchall()

        conn.commit()
        return cur.lastrowid

    finally:
        conn.close()
```

Objective: centralize common database operations.

- `sql` contains the SQL command.
- `params` contains the values used by the command.
- If `fetch=True`, the function returns rows from a SELECT.
- Otherwise, it commits the database change and returns the generated ID when available.
- The `finally` block closes the connection.

Parameterized queries use placeholders such as `%s`:

```
rows = run_query(
    "SELECT id, full_name FROM customers WHERE email = %s",
    (email,),
    fetch=True
)
```

The parameter is passed separately from the SQL text. The comma in `(email,)` makes it a one-element Python tuple.

7 Main Python Classes and Operations

7.1 Customer Class

The customer class stores customer attributes and provides operations to validate, add, update, delete, and retrieve records.

```
class customers:
    def __init__(self, name, email, id=None):
        self.id = id
        self.name = name
        self.email = email

    def validate(self):
        if len(self.name.strip()) < 2:
            raise ValueError(
                "Name must contain at least 2 characters."
            )

        if "@" not in self.email or "." not in self.email.split("@")[-1]:
            raise ValueError("Please enter a valid email address.")

    def add(self):
        self.validate()

        self.id = run_query(
            """INSERT INTO customers
               (full_name, email, created_at)
```

```
            VALUES (%s, %s, %s)""",
            (self.name.strip(), self.email.strip(), date.today())
        )

        return self.id
```

Explanation:

- `__init__()` initializes the customer object.
- `self.id`, `self.name`, and `self.email` store its information.
- `validate()` checks basic input rules.
- `add()` validates the values, inserts the row, and stores the generated customer ID.

MySQL separately enforces the unique email constraint.

7.2 Opening an Account

```
def open(customer_id, initial=Decimal("0.00")):
    conn = get_connection()

    try:
        cur = conn.cursor()
        conn.start_transaction()

        cur.execute(
            """INSERT INTO accounts
               (customers_id, balance, created_at)
               VALUES (%s, %s, %s)""",
            (customer_id, initial, date.today())
        )

        acc_id = cur.lastrowid

        if initial > 0:
            cur.execute(
                """INSERT INTO transactions
                   (account_id, type, amount)
                   VALUES (%s, 'deposit', %s)""",
                (acc_id, initial)
            )

        conn.commit()
        return acc_id

    except Exception:
        conn.rollback()
        raise

    finally:
        conn.close()
```

Objective: create an account for an existing customer.

If the initial amount is positive, an initial deposit is also recorded. The two SQL operations are grouped in one database transaction.

Transaction Principle

`commit()` saves the changes. If an error occurs, `rollback()` cancels the changes made in the transaction.

7.3 Deposit and Withdrawal

The application's shared `_move()` operation updates a balance and inserts the corresponding transaction-history row.

```
cur.execute(
    "SELECT balance FROM accounts WHERE id=%s FOR UPDATE",
    (account_id,)
)

row = cur.fetchone()

if row is None:
    raise ValueError("Account not found.")

```

```
if kind == "withdrawal":
    if row[0] < amount:
        raise ValueError("Insufficient funds.")

    cur.execute(
        "UPDATE accounts SET balance = balance - %s WHERE id=%s",
        (amount, account_id)
    )
else:
    cur.execute(
        "UPDATE accounts SET balance = balance + %s WHERE id=%s",
        (amount, account_id)
    )

cur.execute(
    """INSERT INTO transactions (account_id, type, amount)
       VALUES (%s, %s, %s)""",
    (account_id, kind, amount)
)
```

Explanation:

- `SELECT ... FOR UPDATE` locks the account row during the operation.
- The program checks that the account exists.
- For a withdrawal, it checks the available balance before subtracting the amount.
- It then updates the balance and inserts a history row.
- The full method commits on success and rolls back if an exception occurs.

7.4 Transfers

A transfer changes two account balances and records two transaction rows:

- `transf_out`: records the amount leaving the source account.
- `transf_in`: records the amount entering the destination account.

Transfer Objective

The source debit, destination credit, and two history records must be handled as one database transaction. Either all changes are saved, or none of them are saved.

The implementation also rejects a transfer when the source and destination account IDs are identical, verifies that both accounts exist, and checks that the source account has enough funds.

7.5 Amount Validation

```
def parse_amount(text):
    try:
        value = Decimal(text.strip().replace(",", "."))
    except InvalidOperation:
        raise ValueError("Amount must be a number.")

    if value <= 0:
        raise ValueError("Amount must be greater than 0.")

    return value.quantize(Decimal("0.01"))
```

The `Decimal` type is appropriate for monetary values because it avoids many binary floating-point representation issues.

The function accepts a comma or dot as the decimal separator, rejects invalid or non-positive values, and returns two decimal places.

8 Results: Application in Operation

This section presents screenshots of the GDC BANK application running on its target environment. They illustrate how the modules described in the previous sections appear to the end user.

8.1 Dashboard

The dashboard displays the main application statistics:

- Number of customers and accounts.
- Total account balance.
- Number of transactions made today.
- Seven-day deposits-versus-withdrawals chart.
- Breakdown of money moved by transaction type.
- Top-customers table and recent transactions.

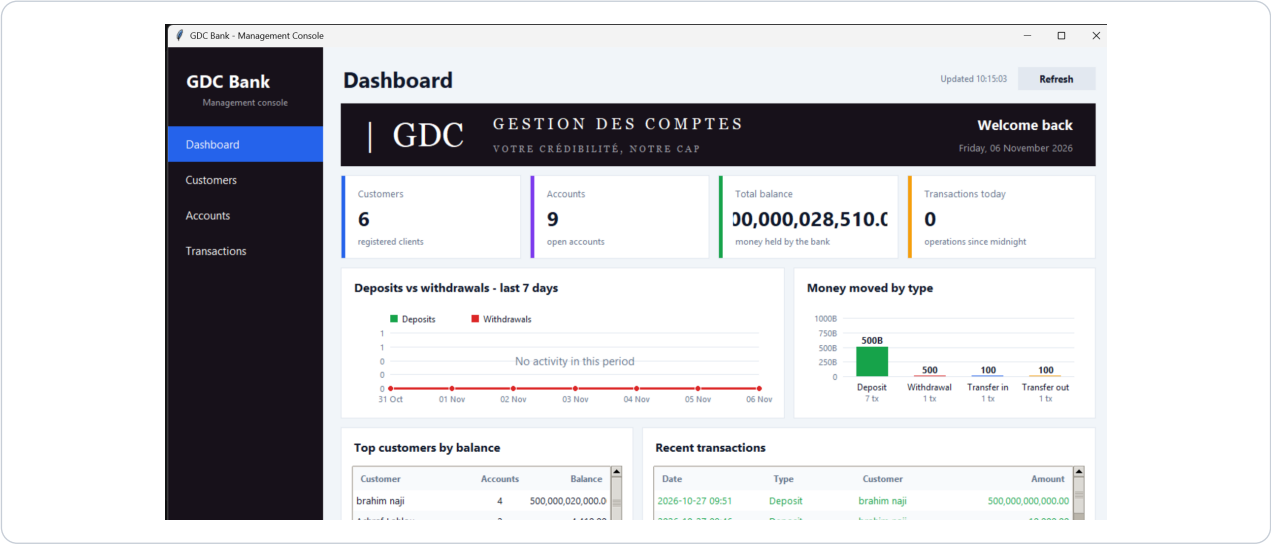


Figure 3: GDC BANK – Dashboard

8.2 Customer Management

The Customers page provides a form for customer information, including full name and email address.

The available actions include Add, Update, Delete, Clear, and Open account for selected. The table displays registered customers, their identifiers, contact information, registration dates, and number of linked accounts. A search field and CSV export are also available.

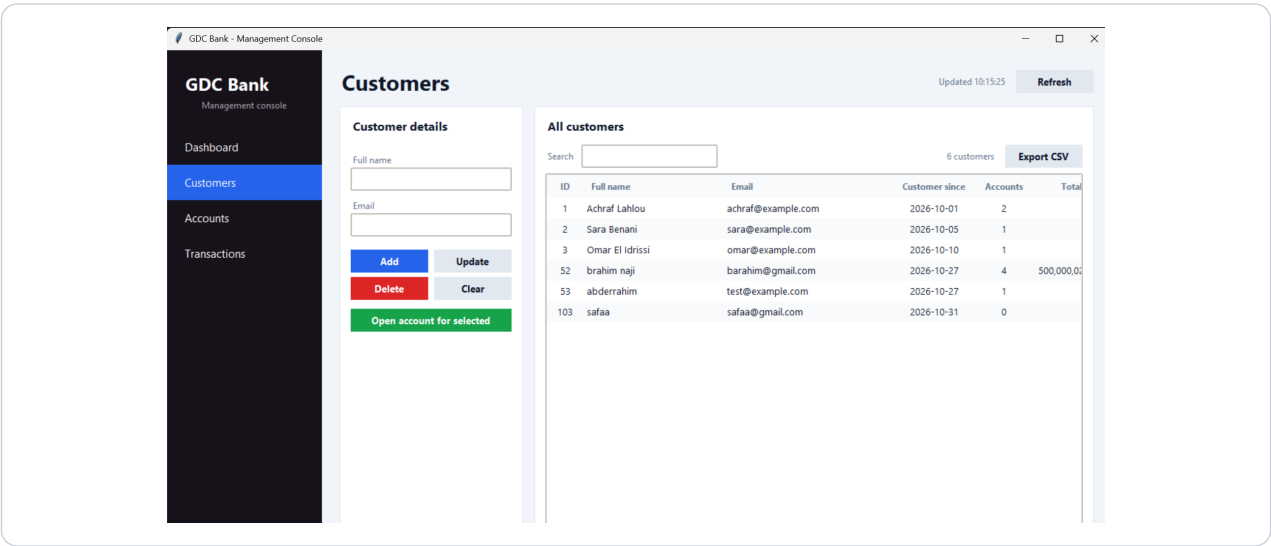


Figure 4: GDC BANK – Customer management

8.3 Account Management

The Accounts page allows the user to select a customer and open a new account with an initial deposit.

The table displays each account, its owner, current balance, and opening date. It also illustrates the relationship in which one customer can own several accounts.

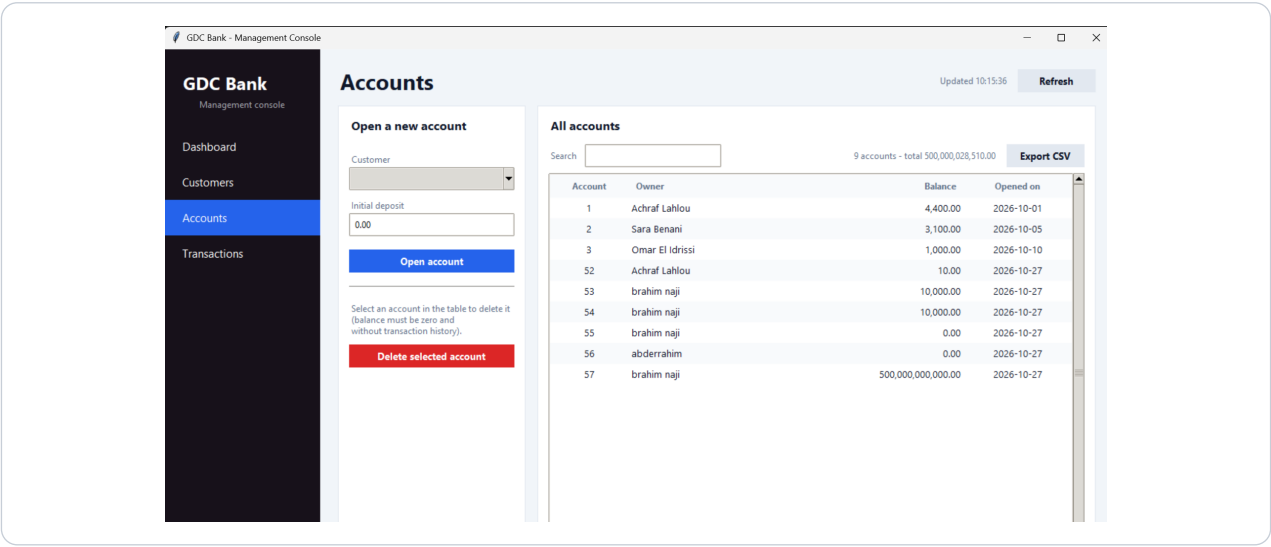


Figure 5: GDC BANK – Account management

8.4 Transaction Management

The Transactions page provides the controls for deposits, withdrawals, and transfers.

The transaction history displays the transaction identifier, account, owner, operation type, amount, and timestamp. These records correspond to the `transactions` table defined in Section 4.

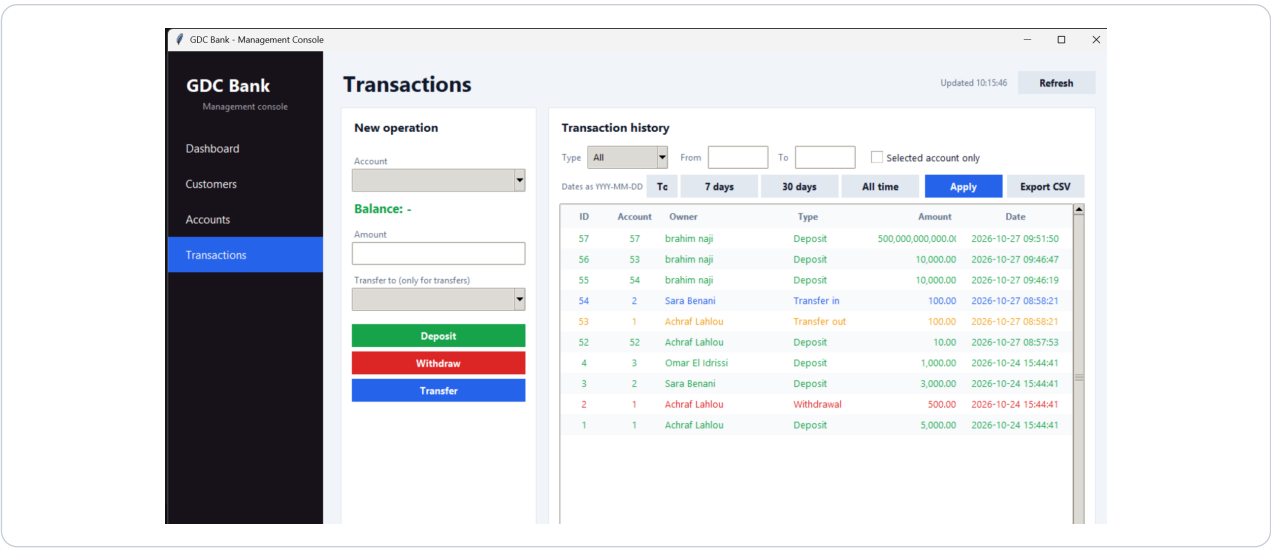


Figure 6: GDC BANK – Transaction history

Observed Result

The screenshots illustrate how the database, Python operations, and Tkinter interface work together. Customers can be associated with multiple accounts, account balances can be updated through simulated transactions, and transaction records are maintained in the database.

9 Conclusion

The GDC BANK project demonstrates the development of a desktop management application using Python, Tkinter, and MySQL.

It brings together customer management, account management, and transaction history in one graphical interface. The project also demonstrates important programming and database concepts, including object-oriented programming, SQL queries, primary and foreign keys, input validation, and database transactions.

Through this project, the team applied theoretical knowledge to a practical application and gained experience in connecting a Python program to a relational database.

Final Perspective

GDC BANK provides an educational example of how a desktop application can organize information, automate management operations, and maintain consistent data through a relational database.

GDC BANK

Python – Tkinter – MySQL

End of Project Documentation